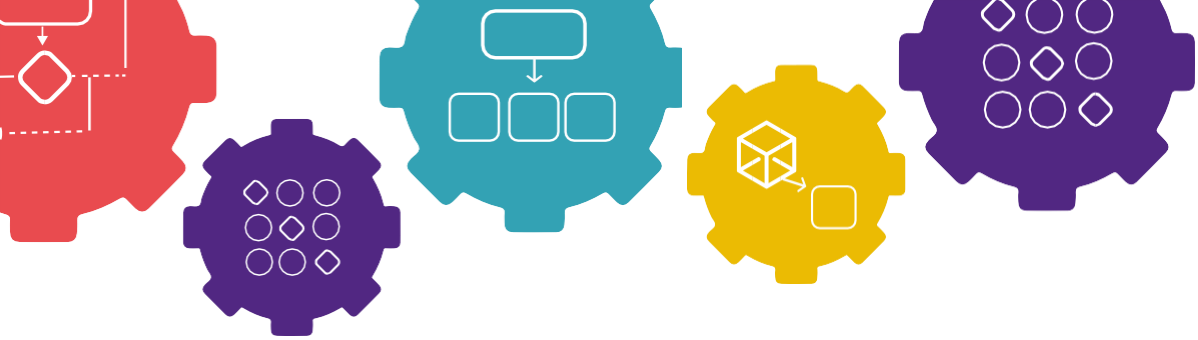


Laboratorio 07

Estructuras Repetitivas

Larissa Méndez 1142526

Enlace del repositorio: https://github.com/Larissa-M008/Laboratorio-PenCom.git

Objetivos

- Implementar algoritmos utilizando estructuras repetitivas (for / while / do-while).
- Implementar estructuras repetitivas anidadas

Conceptos calve

Estructura repetitiva (for / while / do-while):

- Permite ejecutar un conjunto de instrucciones un cierto número ENDito de veces.
- Debemos especificar al programa:
  - Dónde empieza el bucle.
  - Dónde termina el bucle.
  - Cuántas iteraciones debe realizar.

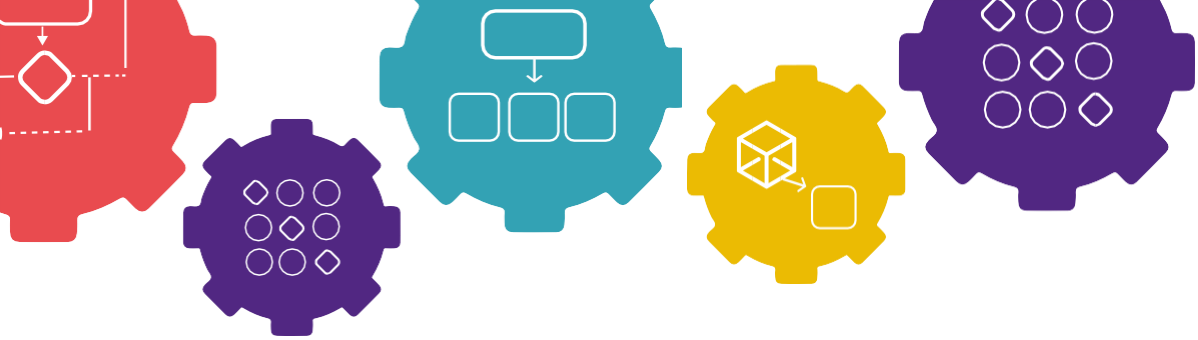
- **Estructuras repetitivas anidada:**
  - Es posible insertar un bucle dentro de otro. La estructura interna debe estar incluida totalmente dentro de la externa y no puede existir solapamiento.
  - Las variables de control toman valores de modo tal que por cada valor de la variable índice del ciclo externo se debe ejecutar totalmente el bucle interno.

Ejercicios

Cree un proyecto nuevo de C#. El nombre del proyecto debe seguir la siguiente sintaxis L7+_+<iniciales>+<carné>. Guárdelo en repositorio creado en el laboratorio anterior y realice los siguientes ejercicios:

Recuerde utilizar la siguiente estructura en su archivo principal

```
using System;
class    Program    {
   static void Main() {
      // Entrada de usuario
      Console.Write("¿Cómo te llamas? ");
      string nombre = Console.ReadLine();
      // Salida de datos
      Console.WriteLine("Hola, " + nombre + " ¡Bienvenido a C#!");
   }
}
```



1. **WHILE**:

   Deberá recrear el siguiente programa, paso por paso, e imprimir en pantalla el resultado final con sus respectivas credenciales:

   Realice un programa que imprima en pantalla, su nombre y carnet. Luego, por medio de un bucle ***while****, deberá mostrar los números del 1 al 20. Los números pares deben mostrarse de color cyan mientras que los impares se mostrarán de color verde.*

   - Primero debe crear las variables que utilizará el programa.

```
//Problema #1 - Mostrar números del 1 al 20.
string nombre = "<Nombre Completo>";
int carnet = 1019126;
int indice = 1;
```

   - Luego muestre en pantalla sus credenciales por medio del comando WriteLine:

```
Console.WriteLine("Nombre: " + nombre + " Carnet: " + carnet.ToString());
```

   - Proceda a crear un bucle **while**, que itere (o se repita) 20 veces. Recuerde que debe alterar el índice dentro del ciclo y colocar entre paréntesis la expresión booleana que indicará cuándo detenerse:

```
while (indice <= 20){ //Mientras indice sea menor o igual a 20

    //Repetiremos las siguientes instrucciones:
    // ---------------
    // ---------------


    //Por ultimo, aumentaremos el indice en 1
    indice = indice + 1;
}
```

   - ¡Perfecto! Ha construido un bucle **while** capaz de “contar hasta 20”. Lo siguiente será mostrar los números del 1 al 20 en pantalla utilizando la variable *índice*.

```
while (indice <= 20){ //Mientras indice sea menor o igual a 20

    //Repetiremos las siguientes instrucciones:
    Console.WriteLine(indice);

    //Por ultimo, aumentaremos el índice en 1
    indice = indice + 1;
}
```

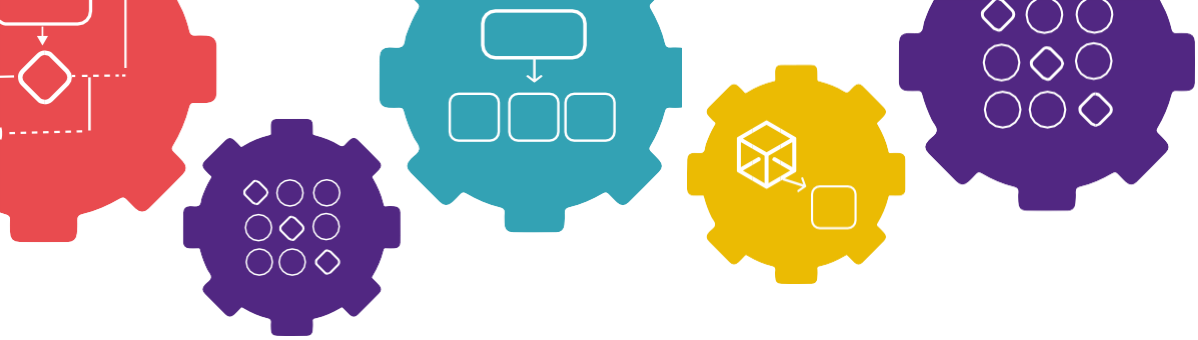
- El problema tiene una condición adicional. Deberá evaluar si el índice a mostrar es par o impar y cambiar el color de la letra respectivamente. Prosiga con la estructura selectiva:

```
while (indice <= 20){

    //Repetiremos las siguientes instrucciones:
    if (indice % 2 == 0) {
        //cambiar el color de la letra a cyan
    }
    else {
        //cambiar el color de la letra a verde
    }
        Console.WriteLine(indice);

    //Por ultimo, aumentaremos el índice en 1
    indice = indice + 1;
}
```

- Utilice el comando *Console.ForegroundColor* para cambiar el color de la letra. En el caso de un índice par asignará el color a Cyan y en el caso de un índice impar se le asignará verde.

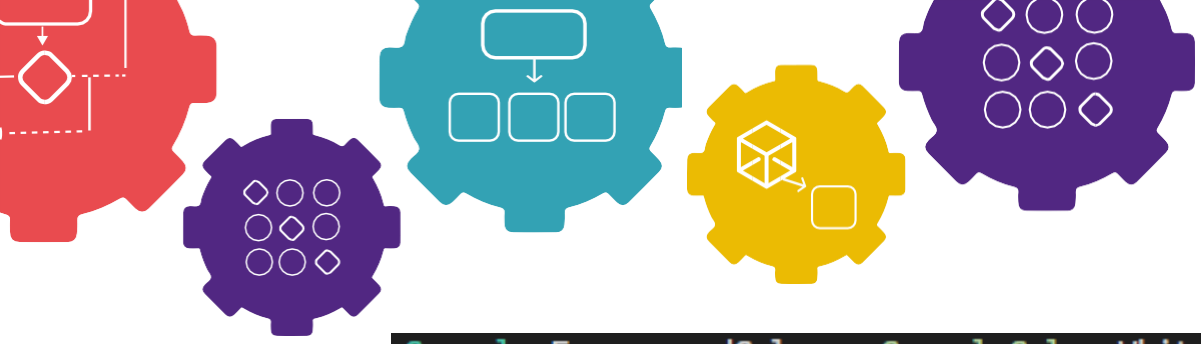


```
while (indice <= 20){

    //Repetiremos las siguientes instrucciones:
    if (indice % 2 == 0) {
        Console.ForegroundColor = ConsoleColor.Cyan;
    }
    else {
        Console.ForegroundColor = ConsoleColor.Green;
    }
        Console.WriteLine(indice);

    //Por ultimo, aumentaremos el índice en 1
    indice = indice + 1;
}
```

- Por último, tras terminar o salir del ciclo, retorne el color de la letra a blanco y ejecute una instrucción ReadLine().



```
Console.ForegroundColor = ConsoleColor.White;
Console.ReadLine();
```

```
string nombre = "Darlene Larissa Méndez Bedoya";
int carnet = 1142526;
int indice = 1;
Console.WriteLine("Nombre: "+nombre+" Carnet: "+carnet.ToString());
while (indice <= 20)
{
    if (indice % 2 == 0)
    {
        Console.ForegroundColor = ConsoleColor.Cyan;
    }
    else
    {
        Console.ForegroundColor = ConsoleColor.Green;
    }

    Console.WriteLine(indice);
    indice = indice + 1;
}
Console.ForegroundColor = ConsoleColor.White;
Console.ReadLine();
```

```
Nombre: Darlene Larissa Méndez Bedoya Carnet: 1142526
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

2. DO-WHILE

Desarrolle un programa en C# que solicite al usuario ingresar un número entero positivo. El programa debe utilizar un ciclo **do-while** para recorrer los valores necesarios y determinar todos los divisores (positivos) del número ingresado. Recuerde que un número es divisor de otro si el residuo de la división es igual a cero.

```
Console.WriteLine();
Console.WriteLine("Ejercicio 2");
Console.WriteLine("Ingrese un número positivo");
int num = int.Parse(Console.ReadLine());
int x = 1;
Console.WriteLine("Sus divisores son:  ");
do {
    if (num % x == 0)
    {
        Console.WriteLine(x);
    }
    x = x + 1;
} while (x<=num);
```

```
Ejercicio 2
Ingrese un número positivo
67
Sus divisores son:
1
67
```

3. FOR

La serie de Fibonacci es una secuencia matemática donde cada número es la suma de los dos anteriores, comenzando con 0 y 1. Por ejemplo:

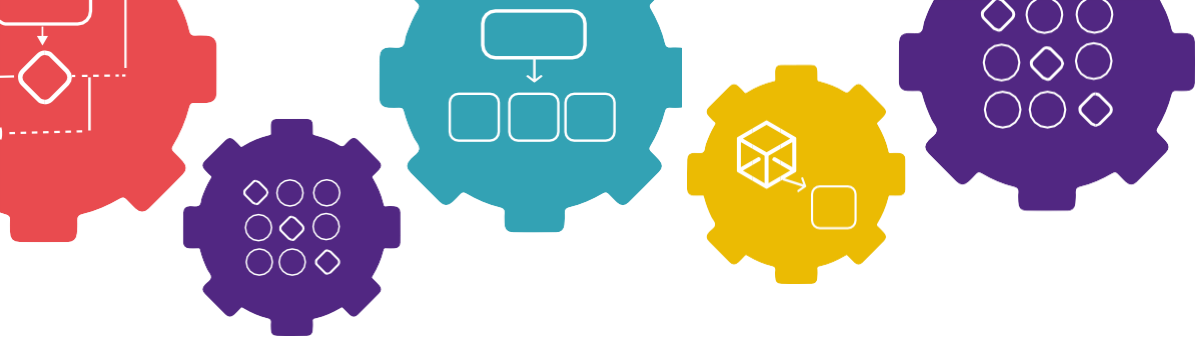
0, 1, 1, 2, 3, 5, 8, 13, 21, etc.

Desarrolle un programa en C# que solicite un número entero “n” al usuario y, por medio de un bucle **for**, imprima en pantalla los primeros n elementos de la serie de Fibonacci.

```
Console.WriteLine();
Console.WriteLine("Ejercicio 3");
Console.WriteLine("Ingrese la posición del número de la secuencia de Fibonacci: ");
int num2 = int.Parse(Console.ReadLine());
Console.WriteLine();
int c = 1;
int b = 0;
for (int i = 0; i < num2; i++)
{
    Console.WriteLine(b +" ");
    int resultado = b;
    b = c;
    c = resultado + c;
}
```

```
Ejercicio 3
Ingrese la posición del número de la secuencia de Fibonacci:
7

0
1
1
2
3
5
8
```



4. CICLOS ANIDADOS – LIBRE

Desarrolle un programa en C# que muestre en pantalla las tablas de multiplicar del 1 al 12 por medio de **ciclos anidados**. Para cada número (del 1 al 12) se debe mostrar un encabezado y sus tablas de multiplicar del 1 al 10. A continuación se muestra un ejemplo de la salida esperada:

```
Console.WriteLine();
Console.WriteLine("Ejercicio 4");
for (int y = 1; y<=12 ; y++)
{
    Console.WriteLine("Tabla del " + y);
    for (int j = 1; j <=10 ; j++)
    {
        Console.WriteLine(y + " * " + j + " = " + (y * j));
    }
    Console.WriteLine("");
}
```

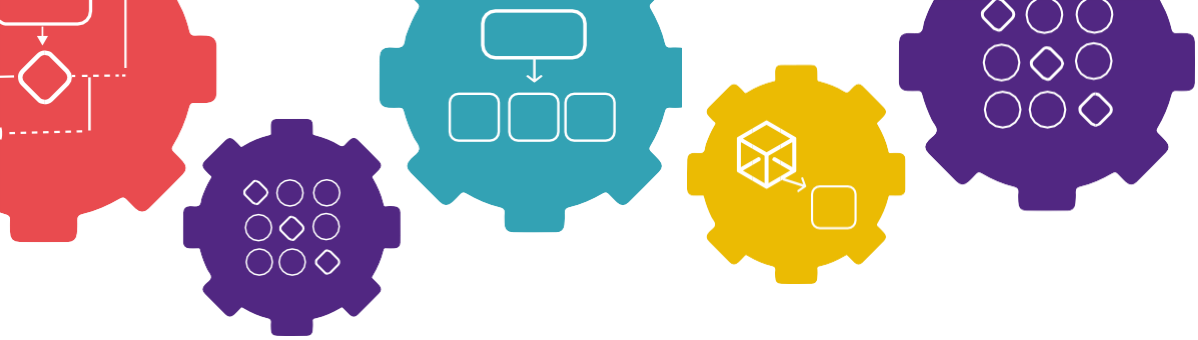
```
Ejercicio 4
Tabla del 1
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10

Tabla del 2
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20

Tabla del 3
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30

Tabla del 4
4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20
4 * 6 = 24
4 * 7 = 28
4 * 8 = 32
4 * 9 = 36
4 * 10 = 40
```

* Sí llega al 12, solo que es muy largo.



	Criterio	Puntaje	Total
Ejercicio 1	Muestra correctamente el resultado en pantalla	10 pts	**10 ptos**
Ejercicio 2	Solicita correctamente un número entero al usuario.	4 pts	**25 ptos**
	Aplica obligatoriamente un ciclo do-while. (de 1 hasta n)	4 pts	
	El contador se actualiza evitando ciclos infinitos	4 pts	
	Utiliza correctamente el operador módulo (%)	4 pts	
	Emplea correctamente estructuras selectivas	4 pts	
	Muestra en pantalla todos los divisores (positivos).	5 pts	
Ejercicio 3	Solicita correctamente un número entero al usuario	5 pts	**25 ptos**
	Aplica obligatoriamente un ciclo for.	5 pts	
	Controla correctamente el número de iteraciones (exactamente N elementos)	5 pts	
	Calcula correctamente cada nuevo término como la suma de los dos anteriores	5 pts	
	Muestra los elementos correctamente en pantalla.	5 pts	
Ejercicio 4	Utiliza dos ciclos correctamente estructurados (uno externo y uno interno)	8 pts	**40 ptos**
	El ciclo externo recorre correctamente las tablas requeridas (1-12)	7 pts	
	El ciclo interno recorre correctamente los multiplicadores (1-10)	7 pts	
	Muestra el encabezado de cada tabla	6 pts	
	Realiza correctamente la operación de multiplicación.	6 pts	
	Presenta la multiplicación en formato legible (ej. 2 x 3 = 6)	6 pts	
Total			**100 ptos**